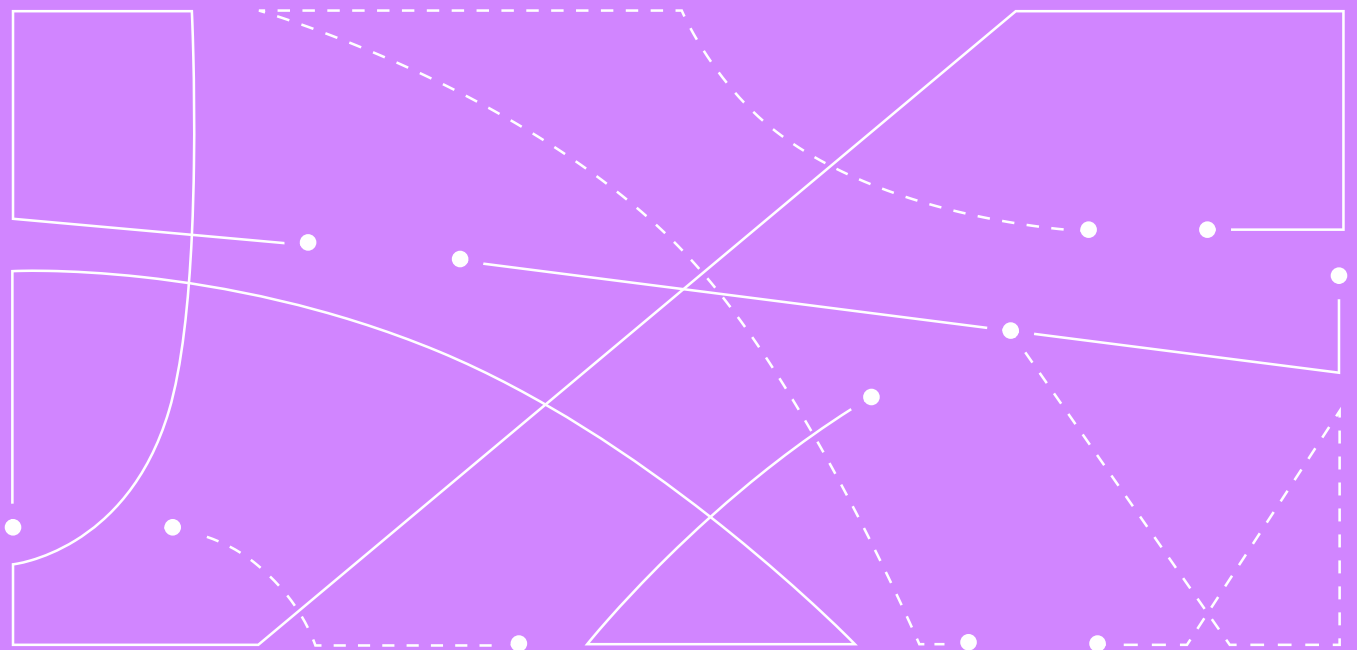




ЦЕНТРАЛЬНЫЙ
УНИВЕРСИТЕТ

Оператор JOIN и теоретико- множественные операции



SQL И БАЗЫ ДАННЫХ (ДЛЯ РАЗРАБОТЧИКОВ)

ЛОНГРИД

НЕДЕЛЯ 4

Введение

Когда необходимо решить небольшую бизнес-задачу и сделать выгрузку из конкретной таблицы, достаточно написать простой запрос, используя предложения SELECT и FROM. Но что, если задача немного сложнее и, чтобы её выполнить, нам необходимо обратиться к двум или более таблицам?

Конечно, можно написать несколько последовательных запросов, использовать результат первого, чтобы написать второй запрос, и так далее. Но будем честны, такой вариант не очень удобный.

В данном лонгриде мы познакомимся с соединением данных и операциями над множествами в SQL и научимся решать задачи, завязанные на нескольких таблицах, одним запросом.

Соединение таблиц

ОСНОВНЫЕ ВИДЫ СОЕДИНЕНИЙ В SQL

Операция соединения является одной из самых важных при работе с реляционными системами баз данных. При помощи соединения можно сопоставить строки одной таблицы со строками другой. В SQL для этого существует предложение JOIN.

Синтаксис

```
SELECT {поля, которые выводим}
FROM {таблица1}
JOIN {таблица2}
ON таблица1.поле_соединения = таблица2.поле_соединения
```

Если поля соединения в двух таблицах называются одинаково, то вместо ON можно использовать USING.

```
SELECT {поля, которые выводим}
FROM {таблица1}
JOIN {таблица2}
USING (поле_соединения)
```

Мы познакомились с общим синтаксисом соединений в SQL, однако существует несколько видов соединения таблиц — давай познакомимся с основными из них.

ВНУТРЕННИЕ СОЕДИНЕНИЯ

INNER JOIN

Начать стоит с внутреннего соединения. Данный тип соединения является самым распространённым. Если коротко, то он позволяет находить пары строк, которые соединяются и удовлетворяют предикату соединения. Проще говоря, результат запроса с использованием внутреннего соединения — строки, которые есть как в первой, так и во второй таблице.

Синтаксис

```
SELECT {поля, которые выводим}
FROM {таблица1}
INNER JOIN {таблица2}
ON таблица1.поле_соединения = таблица2.поле_соединения
```

Для того чтобы лучше разобраться с соединениями, введём две тестовые таблицы, на которых будем разбираться.

Таблица А

client_id	name	gender_cd
1	Макс	M
2	Наташа	F
3	Никита	M

Таблица В

client_id	profession
4	Продакт-менеджер
2	Аналитик
3	Инженер данных

ПРИМЕР

Определить профессии всех сотрудников

```
SELECT *
FROM Таблица А a
INNER JOIN Таблица В b
ON a.client_id = b.client_id
```

Результат

a.client_id	a.name	gender_cd	b.client_id	b.profession
2	Наташа	F	2	Аналитик
3	Никита	M	3	Инженер данных

Можем заметить, что остались только строки, содержащие информацию о втором и третьем клиентах, так как по ним есть данные в обеих таблицах. Данные о первом клиенте есть только в таблице A, а о четвёртом клиенте есть только в таблице B.

ПРИМЕР

Определить профессию всех клиенток

```
SELECT *  
FROM "Таблица A" a  
    INNER JOIN "Таблица B" b  
    ON a.client_id = b.client_id  
WHERE gender_cd = 'F'
```

Результат

a.client_id	a.name	gender_cd	b.client_id	b.profession
2	Наташа	F	2	Аналитик

В запросе выше мы ищем пересечение клиентов с профессиями, соединяя их по ID клиента, после чего оставляем только клиентов-женщин.

ВАЖНО

- Если в запросе записать просто JOIN, то будет выполнен INNER JOIN.
- После ON указываются условия — по каким полям будет происходить соединение. Можно проводить соединение по нескольким полям (для этого дополнительные условия нужно добавить через AND/OR). С примерами таких запросов познакомимся чуть позже, когда будем разбирать соединения по неравенству.
- Внутренние соединения полностью коммутативны. A INNER JOIN B и B INNER JOIN A эквивалентны.

Также важно заметить, что если после **SELECT** указать *****, желая вывести все поля, то при использовании соединений будут выведены столбцы из обеих таблиц. Так как в таблицах существуют одинаковые столбцы, выводя абсолютно все столбцы, мы столкнёмся с рядом ошибок.

Чтобы их избежать, лучше придерживаться следующих правил:

- Выводить только необходимые столбцы из первой или второй таблицы.
- Указывать алиасы при выборе полей. Если в случае вывода столбца, который представлен в нескольких таблицах, не указать алиас, запрос упадёт в ошибку.

ПЕРЕКРЁСТНЫЕ СОЕДИНЕНИЯ

CROSS JOIN

Иногда возникает задача, в которой необходимо получить все уникальные пары строк из двух таблиц — говоря более математическим языком, выполнить декартово произведение двух таблиц. По итогам данного соединения каждая строка первой таблицы будет соединена с каждой строкой второй таблицы.

Синтаксис

```
SELECT *  
FROM "Таблица А" a  
    CROSS JOIN "Таблица В" b  
  
-- ИЛИ  
  
SELECT *  
FROM "Таблица А", "Таблица В"
```

Для чего это нужно на практике? Например, у нас может быть таблица клиентов и таблица всех календарных дат. Мы преследуем цель посчитать определённую метрику для каждого клиента на каждую дату. Чтобы выполнить данную задачу, хотим получить все возможные уникальные пары клиентов и дат.

Данный тип соединения называется перекрёстным, в SQL записывается как CROSS JOIN. Важно отметить, что для перекрёстного соединения нельзя определить предикат соединения, используя для этого предложение ON, поэтому прописывать, по какому полю идёт соединение, не нужно.

Введём две новые таблицы.

Таблица дат

Дата
2024-01-01
2024-02-01
2024-03-01

Таблица клиентов

Клиент
Вася
Петя
Егор

ПРИМЕР	Соединим с помощью CROSS JOIN таблицу дат и клиентов	
	<pre>SELECT * FROM таблица_дат CROSS JOIN таблица_клиентов</pre>	
	Результат	
	Дата	Дата
	2024-01-01	Вася
	2024-02-01	Вася
	2024-03-01	Вася
	2024-01-01	Петя
	2024-02-01	Петя
	2024-03-01	Петя
	2024-01-01	Егор
	2024-02-01	Егор
	2024-03-01	Егор

Перекры́стные соединения на практике используются довольно редко. Не стоит без необходимости пересекать две большие таблицы, поскольку это задействует очень дорогие операции и получится очень большой результирующий набор данных, с которым будет тяжело работать.

ВНЕШНИЕ СОЕДИНЕНИЯ

Однако иногда возникает задача, когда нам нужно вывести всю информацию из одной из таблиц, а также добавить (по возможности) информацию из другой таблицы. Данный тип соединения называется внешним.

Существует три вида внешних соединений:

- **LEFT OUTER JOIN,**
- **RIGHT OUTER JOIN,**
- **FULL OUTER JOIN.**

Разберём на примерах.

LEFT JOIN

Синтаксис

```
SELECT {поля из таблиц}
FROM {первая таблица} a
LEFT JOIN {вторая таблица} b
ON a.{поле соединения} = b.{поле соединения}
```

Работает данный вид соединения следующим образом: сохраняем всё из первой таблицы, добавляем данные из второй. В случае если во второй таблице не нашлось информации, поля заполняются значением NULL.

ПРИМЕР	Соединим с помощью LEFT JOIN тестовые таблицы				
	<pre>SELECT * FROM "Таблица А" a LEFT JOIN "Таблица В" b ON a.client_id = b.client_id</pre>				
	Результат				
	a.client_id	a.name	a.gender_cd	b.client_id	b.profession
	1	Макс	M	NULL	NULL
2	Наташа	F	2	Аналитик	
3	Никита	M	3	Инженер данных	
Можем заметить, что в результате запроса есть информация обо всех клиентах таблицы А, но для первого клиента новые столбцы заполнены пропусками, так как о нём нет информации в таблице В.					

RIGHT JOIN

Синтаксис

```
SELECT {поля из таблиц}
FROM {первая таблица} a
RIGHT JOIN {вторая таблица} b
ON a.{поле соединения} = b.{поле соединения}
```

Работает данный вид соединения следующим образом: выводим все данные уже из второй таблицы, добавляем, где есть, данные из первой, остальное заполняем NULL.

ПРИМЕР	Соединим с помощью RIGHT JOIN тестовые таблицы				
	<pre>SELECT * FROM "Таблица А" a RIGHT JOIN "Таблица В" b ON a.client_id = b.client_id</pre>				
	Результат				
	a.client_id	a.name	a.gender_cd	b.client_id	b.profession
	NULL	NULL	NULL	4	Продакт-менеджер
	2	Наташа	F	2	Аналитик
	3	Никита	M	3	Инженер данных
Можем заметить, что в результате запроса есть информация обо всех клиентах таблицы В, но имя четвёртого клиента мы не смогли узнать, так как про него нет информации в таблице А.					

FULL JOIN

Синтаксис

```
SELECT {поля из таблиц}
FROM {первая таблица} a
      FULL JOIN {вторая таблица} b
      ON a.{поле соединения} = b.{поле соединения}
```

Работает данный вид соединения следующим образом: выводит все строки как первой, так и второй таблицы. Если чего-то не хватает — заполняет значением NULL

ПРИМЕР	Соединим с помощью FULL JOIN тестовые таблицы				
	<pre>SELECT * FROM "Таблица А" a FULL JOIN "Таблица В" b ON a.client_id = b.client_id</pre>				

Результат				
a.client_id	a.name	a.gender_cd	b.client_id	b.profession
NULL	NULL	NULL	4	Продакт-менеджер
2	Наташа	F	2	Аналитик
3	Никита	M	3	Инженер данных
1	Макс	M	NULL	NULL

В результате получили информацию обо всех клиентах, встречающихся хотя бы в одной из таблиц. Все недостающие данные заполнены пропуском.

Также стоит отметить, что полные внешние соединения коммутативны.

Разберём ещё один пример на внешние соединения.

ПРИМЕР	Вывести всех клиентов-мужчин и, если есть, добавить информацию об их профессии															
	<pre>SELECT * FROM "Таблица A" a LEFT JOIN "Таблица B" b ON a.client_id = b.client_id WHERE gender_cd = 'M' -- ИЛИ SELECT * FROM "Таблица B" b RIGHT JOIN "Таблица A" a ON a.client_id = b.client_id WHERE gender_cd = 'M'</pre>															
	Результат <table><tr><th>a.client_id</th><th>a.name</th><th>a.gender_cd</th><th>b.client_id</th><th>b.profession</th></tr><tr><td>1</td><td>Макс</td><td>M</td><td>NULL</td><td>NULL</td></tr><tr><td>3</td><td>Никита</td><td>M</td><td>3</td><td>Инженер данных</td></tr></table> Данную задачу можно решить двумя способами: используя RIGHT JOIN (указываем таблицу B второй) или LEFT JOIN (указываем таблицу B первой). Результаты будут эквивалентны.	a.client_id	a.name	a.gender_cd	b.client_id	b.profession	1	Макс	M	NULL	NULL	3	Никита	M	3	Инженер данных
a.client_id	a.name	a.gender_cd	b.client_id	b.profession												
1	Макс	M	NULL	NULL												
3	Никита	M	3	Инженер данных												

ВАЖНОЕ ПРО JOIN

Множественные JOIN

Первое, что хотелось бы подсветить в данном разделе, — это множественные JOIN. Они позволяют объединять данные из нескольких таблиц в один результирующий набор. Это мощный инструмент для анализа сложных данных, однако он требует тщательного планирования и понимания различных типов JOIN.

Когда использовать множественные JOIN:

- Для анализа взаимосвязей между данными из нескольких таблиц — например, чтобы сделать анализ продаж по регионам и продуктам.
- Чтобы создавать отчёты, объединяющие информацию из различных источников, — например, для создания отчёта о клиентах, содержащего их контактную информацию, историю заказов и данные о доставке.

Синтаксис

```
SELECT {поля, которые выводим}
FROM {таблица1}
    (INNER / LEFT / RIGHT) JOIN {таблица2}
    ON таблица1.поле_соединения = таблица2.поле_соединения
    (INNER / LEFT / RIGHT) JOIN {таблица3}
    ON таблица1.поле_соединения = таблица3.поле_соединения
    (INNER / LEFT / RIGHT) JOIN {таблица4}
    ON таблица2.поле_соединения = таблица4.поле_соединения
```

Чтобы перейти к задачам, давай добавим ещё одну таблицу — с заказами клиентов.

Таблица C

order_id	client_id	order_date	order_sum
1	1	2024-01-01	500
2	2	2024-02-12	1000
3	3	2024-03-19	2000

ПРИМЕР

Рассчитай сумму заказов клиентов из таблицы А. Если клиент не совершал заказы, то информацию по нему выводить не нужно

В ответе вывести:

- ID клиента;
- имя клиента;
- профессию клиента (если имеется);
- сумму всех заказов.

```
SELECT
    a.client_id
    , a.name
    , b.profession
    , sum(c.order_sum) as sum_
FROM "Таблица А" a
    LEFT JOIN "Таблица В" b
        ON a.client_id = b.client_id
    INNER JOIN "Таблица С" c
        ON a.client_id = b.client_id
GROUP BY
    a.client_id
    , a.name
    , b.profession
```

Результат

a.client_id	a.name	b.profession	sum_
1	Макс	NULL	500
3	Наташа	Аналитик	3000

В примере выше мы сначала соединили таблицу клиентов с таблицей профессий. Для этого использовали LEFT JOIN, так как клиенты без информации о профессии нам тоже интересны. После чего соединили промежуточный результат с таблицей заказов. Использовали INNER JOIN, так как клиенты без заказов нам неинтересны.

JOIN по неравенству

Следующий важный момент — возможность производить JOIN по неравенству. Во всех примерах ранее в условии соединения мы прописывали равенство. Однако для ряда задач может пригодиться JOIN по неравенству.

```

SELECT {поля, которые выводим}
FROM {таблица1}
    (INNER / LEFT / RIGHT) JOIN {таблица2}
    ON таблица1.поле_соединения >= таблица2.поле_соединения

```

-- ИЛИ

```

SELECT {поля, которые выводим}
FROM {таблица1}
    (INNER / LEFT / RIGHT) JOIN {таблица2}
    ON таблица1.поле_соединения <= таблица2.поле_соединения

```

ПРИМЕР

Вывести всех клиентов (ID клиентов), которые совершали заказ после 1 марта 2024 года

```

SELECT o1.client_id
FROM "Таблица А" o1
    INNER JOIN "Таблица В" o2
    ON o1.client_id = o2.client_id
    AND o2.order_date >= '2024-03-01'

```

Результат

client_id

2

Использование одной и той же таблицы в join несколько раз

Также при необходимости можно соединяться с одной и той же таблицей несколько раз, иногда такое соединение называют SELF JOIN. Данный вид не является отдельным предложением в SQL, SELF JOIN можно записать как через внутреннее соединение, так и через, например, LEFT JOIN. Основная идея заключается в связке таблицы с самой собой.

```

SELECT *
FROM Таблица A a
    (INNER / LEFT / RIGHT) JOIN Таблица A b
    ON {условие соединения}

```

ПРИМЕР

Вывести ID заказа, ID клиента и дату заказа, а также аналогичную информацию о предыдущих заказах данного клиента

```

SELECT o1.order_id
      , o1.client_id
      , o1.order_date
      , o2.order_id as order_id2
      , o2.client_id as client_id2
      , o2.order_date as order_date2
FROM "Таблица C" o1
     INNER JOIN "Таблица C" o2
     ON o1.client_id = o2.client_id
     AND o1.order_date > o2.order_date

```

Результат

order_id	client_id	order_date	order_id2	client_id2	order_date2
3	2	2024-03-19	2	2	2024-02-12

Порядок выполнения запросов с JOIN

Также важно понимать порядок выполнения предложения JOIN в запросе. Мы уже знаем, что порядок выполнения запроса без использования соединений следующий:

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

Если добавить в данную последовательность JOIN, то получим следующую картину:

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

ИТОГОВАЯ ТАБЛИЦА СО ВСЕМИ ИЗУЧЕННЫМИ ВИДАМИ СОЕДИНЕНИЯ

Соединение	Результат
A INNER JOIN B	К строкам в таблице A присоединит строки из таблицы B. Если соответствие не было найдено, то строки из A (или из B) не будут выведены
A LEFT OUTER JOIN B	Выведет все строки таблицы A, к ним добавит строки из таблицы B (где это соответствие было найдено). Если соответствие не было найдено, то заполнит все столбцы из таблицы B пропусками
A RIGHT OUTER JOIN B	Выведет все строки таблицы B, к ним добавит строки из таблицы A (где это соответствие было найдено). Если соответствие не было найдено, то заполнит все столбцы из таблицы A пропусками
A FULL OUTER JOIN B	Выведет все строки из A и B. Те, которым не нашли соответствие, заполнит пропусками
A CROSS JOIN B	Все возможные комбинации строк из A и B

Операции над множествами

Зачастую в практике можно столкнуться со следующими задачами:

- объединить результаты из нескольких таблиц с одинаковой структурой;
- найти общие элементы двух наборов;
- найти различия между наборами;
- подготовить «сводку» из разных источников без джойнов (когда связи между таблицами нет или не нужна).

Для этого в SQL есть операции над множествами: UNION [ALL], INTERSECT [ALL], EXCEPT [ALL].

Далее разберём синтаксис, примеры, продвинутые случаи с сортировкой/фильтрацией/группировкой, особенности производительности и типовые ошибки.

ЧТО ТАКОЕ ОПЕРАЦИИ НАД МНОЖЕСТВАМИ

Операции над множествами объединяют результаты двух и более запросов (каждый запрос — это «множество»).

- UNION — объединение двух наборов данных с неявной дедубликацией, то есть результат операции даст уникальные строки.
- UNION ALL — объединение с сохранением дублей.
- INTERSECT — пересечение, операция даёт уникальные строки, присутствующие в обеих выборках.

- EXCEPT — разность, операция даёт уникальные строки из первой выборки, которых нет во второй.
- Варианты INTERSECT ALL и EXCEPT ALL учитывают кратность строк, но используются крайне редко.

Основные требования к множествам (таблицам или заборсам)

- В каждом запросе или таблице должен быть одинаковый набор столбцов (по количеству).
- Соответствующие столбцы должны быть совместимых типов (чтобы СУБД смогла привести их к общему типу).
- Алиасы столбцов берутся из первого запроса или таблицы.

Базовый синтаксис

```

SELECT ...           -- множество 1
FROM ...
[WHERE ...]
[GROUP BY ...]
[HAVING ...]
[ORDER BY ...]      -- действует только в пределах подзапроса,
если есть LIMIT (см. ниже)
UNION | UNION ALL | INTERSECT | EXCEPT
SELECT ...           -- множество 2
FROM ...
...

```

Чтобы сортировать/фильтровать общий результат, оборачиваем всю операцию в подзапрос и пишем ORDER BY / WHERE / GROUP BY снаружи (подробно — ниже).

Часто операции над множествами нужны с удалением дублей. Вот несколько простых примеров

ПРИМЕР	UNION			
	UNION — все уникальные строки из двух множеств.			
	Исходные данные			
	w_country		w_city	
	name	pop	name	pop
	France	67M	Paris	2M
	Spain	47M	Madrid	3M

Italy	60M	Rome	2.8M
Andorra	0.08M	Andorra	0.02M

SQL

```

SELECT name AS geo_name, 'country' AS geo_type
FROM w_country

UNION

SELECT name AS geo_name, 'city' AS geo_type
FROM w_city;

```

Результат

geo_name	geo_type
France	country
Spain	country
Italy	country
Andorra	country
Paris	city
Madrid	city
Rome	city
Andorra	city

ПРИМЕР

INTERSECT

INTERSECT — только те строки, что есть в обоих множествах (уникальные).

Найти страны и города с одинаковым названием.

Исходные данные

w_country		w_city	
name	pop	name	pop
Paris	2M	France	67M
Andorra	0.02M	Andorra	0.08M
Rome	2.8M	Italy	60

SQL

```
SELECT name FROM w_city

INTERSECT

SELECT name FROM w_country;
```

Результат

name

Andorra

ПРИМЕР

EXCEPT

EXCEPT — строки из первой выборки, которых нет во второй (уникальные)

Найти страны, у которых нет города с таким же названием.

Исходные данные

w_country		w_city	
name	pop	name	pop
France	67M	Paris	2M
Spain	47M	Madrid	3M
Andorra	0.08M	Andorra	0.02M

SQL
<pre>SELECT name FROM w_country EXCEPT SELECT name FROM w_city;</pre>
Результат
name
France
Spain

ОПЕРАЦИИ С СОХРАНЕНИЕМ ДУБЛЕЙ, НУЖНЫ РЕЖЕ И В ДОВОЛЬНО СПЕЦИФИЧНЫХ СЛУЧАЯХ. ВОТ НЕСКОЛЬКО ПРИМЕРОВ

Ниже три примера, где вариант **ALL** даёт смысловой эффект, где кратность вхождения дублей действительно важна.

UNION ALL: конкатенация дневных логов кликов (без удаления дублей)

Идея. У нас есть логи кликов за два дня. Одинаковые строки возможны: повторная отправка события со стороны клиента/SDK, ретраи, дубликаты со стороны брокера. Нам важно сохранить **все** события (в том числе дубликаты).

Исходные данные

events_ day1				events_ day2			
event_ id	user_id	page	ts	event_ id	user_id	page	ts
e101	u1	/plp	10:01	e201	u2	/plp	11:02
e102	u1	/pdp	10:03	e202	u1	/pdp	11:05
e103	u2	/plp	10:07	e203	u1	/pdp	11:05 (полный дубль)
e104	u3	/home	10:10	e204	u3	/cart	11:06

```

SELECT event_id, user_id, page, ts
FROM events_day1

UNION ALL

SELECT user_id, page
FROM events_day2;

```

Результат

event_id	user_id	page	ts
e101	u1	/plp	10:01
e102	u1	/pdp	10:03
e103	u2	/plp	10:07
e104	u3	/home	10:10
e201	u2	/plp	11:02
e202	u1	/pdp	11:05
e203	u1	/pdp	11:05
e204	u3	/cart	11:06



Зачем ALL? Нам нельзя терять повторные события — иначе сожмём метрики, где нужны «сырые» клики (например, мониторинг ретраев/дубликатов SDK).

INTERSECT ALL: «повторяющиеся покупки» по месяцам (учитываем кратность)

INTERSECT ALL учитывает кратность: строка вернётся столько раз, сколько она встречается минимально в обеих выборках.

Идея. Смотрим товары, которые покупал пользователь **и в январе, и в феврале**. Если товар взяли три раза в январе и один раз в феврале, в **INTERSECT ALL** он появится **один раз** — как «минимальная повторяемость». Это удобно для оценки «стабильно повторяющихся» позиций.

Исходные данные

order_items_jan				order_items_feb			
user_id	product_id	qty	ts	user_id	product_id	qty	ts
u1	A	1	01	u1	A	1	02
u1	A	1	05	u1	A	1	14
u1	B	1	07	u1	B	1	20
u1	A	1	21	u1	C	1	22



Для простоты считаем поштучные покупки (1 строка = 1 единица).
Так кратность формируется **строками**.

SQL

```
SELECT user_id, product_id
FROM order_items_jan

INTERSECT ALL

SELECT user_id, product_id
FROM order_items_feb;
```

Результат

user_id	product_id
u1	A
u1	A
u1	B



Почему ALL? Без **ALL** пересечение вернуло бы просто {A, B}.
С **ALL** мы видим «насколько устойчиво» повторялась покупка
(минимальная кратность между месяцами).

EXCEPT ALL: поставки минус возвраты (сколько единиц осталось «в деле»)

EXCEPT ALL вычитает строки с учётом кратности.

Идея. Каждый юнит товара идёт отдельной строкой в поставках и в возвратах (возврат — это вычитание одной единицы). Если поставили три штуки и вернули одну, в **EXCEPT ALL** останутся две строки.

Исходные данные

shipments (поставки)			returns (возвраты)		
sku	serial	ship_dt	sku	serial	ship_dt
A	s1	2024-03	A	s1	2024-03
A	s1	2024-03	B	t1	2024-03
A	s1	2024-03			
B	t1	2024-03			

SQL

```
SELECT sku, serial
FROM shipments

EXCEPT ALL

SELECT sku, serial
FROM returns;
```

Результат

sku	serial
A	s1
A	s1



Почему ALL? Если бы мы использовали Except, то получилось бы пустое множество, хотя вернули только один товар из трёх в A, s1.

Итог

- **UNION ALL** — конкатенирует данные, **сохраняя** дубликаты (сырые логи/события).
- **INTERSECT ALL** — оставляет общие элементы, но **с кратностью** (минимум повторов между выборками).
- **EXCEPT ALL** — вычитает, **сохраняя кратность** (показывает, что реально «осталось» после возвратов/отмен).

UNION И UNION ALL — ЧТО ВЫБРАТЬ

- UNION убирает дубликаты (неявная дедупликация).
- UNION ALL оставляет все строки.

UNION ALL МОЖЕТ БЫТЬ БЫСТРЕЕ, ДАЖЕ ЕСЛИ ДУБЛИКАТЫ НЕ НУЖНЫ

UNION вынужден сортировать/хешировать результат для удаления дублей → дополнительная память и CPU.

UNION ALL просто «склеивает» потоки без дополнительных этапов.

Часто разумнее собрать всё через UNION ALL, а если нужна дедупликация, сделать её позже и только там, где надо (например, при финальной агрегации или SELECT DISTINCT уже над узким набором столбцов).

Также, если уникальность гарантирована, например источники не пересекаются по ключу, лучше использовать UNION ALL, так как это быстрее.

Приём: когда в финале всё равно считаешь агрегаты, используй UNION ALL и агрегируй.

ПРИМЕР	Сумма продаж по категориям из двух источников
	<pre>SELECT category_id, SUM(amount) AS total_amount FROM (SELECT category_id, amount FROM shop_sales_2024q1 UNION ALL SELECT category_id, amount FROM shop_sales_2024q2) t GROUP BY category_id;</pre>
	Здесь дедупликация не нужна — мы всё равно делаем группировку на финальном шаге.

→ Когда ты работаешь с большими объёмами данных, то разумнее использовать UNION ALL. Так как в нём нет неявной дедупликации, а значит, работать он будет быстрее. Если же дубли необходимо убрать, это лучше сделать на финальных шагах через DISTINCT или GROUP BY.

ТРЕБОВАНИЯ К ОПЕРАЦИЯМ НАД МНОЖЕСТВАМИ

Одинаковое количество столбцов

Плохо

```
SELECT id, name FROM w_city
UNION
SELECT name FROM w_country; -- ошибка: 2 колонки vs 1
```

Правильно

```
SELECT id, name FROM w_city
UNION
SELECT id, name FROM w_country; -- 2 колонки и там и там
```

Имена/алиасы берутся из первого запроса или таблицы

```
SELECT name AS geo_name FROM w_country
UNION ALL
SELECT name FROM w_city;
-- Итоговая колонка будет называться geo_name
```

СОРТИРОВКА, ФИЛЬТРАЦИЯ И ГРУППИРОВКА С ОПЕРАЦИЯМИ НАД МНОЖЕСТВАМИ

1. Сортировка.

Вариант А: сортируем общий результат.

```
SELECT name AS geo_name, 'country' AS geo_type FROM w_country
UNION ALL
SELECT name AS geo_name, 'city' AS geo_type FROM w_city
ORDER BY geo_name
LIMIT 50;
```

- Сортировка применяется после объединения.
- Работают алиасы из первого запроса.

Вариант Б: сортируем один из запросов + ограничиваем его.

```
(
    SELECT name AS geo_name, 'city' AS geo_type, population
    FROM w_city
    ORDER BY population DESC
    LIMIT 50
)
UNION ALL
SELECT name, 'country', population FROM w_country;
```

- Сортировка и лимит действуют только на первый запрос.
- Полезно, когда нужно «впустить в итог» лишь Top-N/Last-N из одного (или обоих) из запросов.
- Скобки обязательны.



Подзапрос без использования LIMIT часто не гарантирует порядок строк, то есть внутренний ORDER BY без LIMIT может быть оптимизатором проигнорирован как не влияющий на результат множества.

2. Фильтрация.

Фильтровать множество просто, можно без скобок.

```
SELECT ... FROM A WHERE ...
UNION ALL
SELECT ... FROM B WHERE ...;
```

Фильтровать общий результат — только через подзапрос.

```
SELECT *
FROM (
    SELECT id, 'A' AS src FROM A
    UNION ALL
    SELECT id, 'B' AS src FROM B
) t
WHERE src = 'B'           -- фильтр на весь итог
```

(WORLD): оставить только объекты с населением >1 миллиона независимо от типа

Вариант 1:

```
SELECT name AS geo_name, 'country' AS geo_type,
population AS pop
FROM w_country
WHERE pop > 1000000

UNION ALL

SELECT name, 'city', population
FROM w_city
WHERE population > 1000000 -- Если тут написать pop --
будет ошибка
```

Вариант 2:

```
SELECT *
FROM (
    SELECT name AS geo_name, 'country' AS geo_type,
population AS pop
    FROM w_country
    UNION ALL
    SELECT name, 'city', population
    FROM w_city
) t
WHERE pop > 1000000; --Если тут написать population
будет ошибка
```

3. Группировка.

Как и с фильтрацией: группировать общий результат можно только снаружи.

```
SELECT geo_type, COUNT(*) AS cnt
FROM (
    SELECT name AS geo_name, 'country' AS geo_type FROM w_country
    UNION ALL
    SELECT name AS geo_name, 'city' AS geo_type FROM w_city
) t
GROUP BY geo_type;
```

Группировка внутри запроса тоже работает (например, чтобы агрегировать континенты).

```
SELECT continent AS geo_name, 'continent' AS geo_type,
SUM(population) AS pop
FROM w_country
GROUP BY continent

UNION ALL

SELECT name, 'country', population
FROM w_country;
```

ЛУЧШИЕ ПРАКТИКИ

- По умолчанию на больших данных используй UNION ALL — это быстрее, а дедубликацию, если необходимо, оставь на финальную стадию, где она действительно нужна.
- Явно приводи типы с ::cast, особенно если сомневаешься в совместимости.
- Алиасы в первом запросе — сразу задавай понятные имена колонок.

ТИПОВЫЕ ОШИБКИ

1. Неправильный порядок столбцов.

```
-- В первом запросе (name, population), во втором (population, name)

SELECT name, population FROM w_city
UNION
SELECT population, name FROM w_country;    -- типы по позициям
не совпадают
```

Как надо?

```
SELECT name, population FROM w_city
UNION
SELECT name, population FROM w_country;
```

2. Несовместимые типы данных.

```
SELECT id, created_at          -- created_at: timestamp
FROM flights_current
UNION
SELECT id, departed_flag      -- departed_flag: boolean
FROM flights_archive;         -- ❌ timestamp vs boolean
```

Решение: привести к общему типу.

```
SELECT id, (created_at IS NOT NULL)::boolean AS departed_flag
FROM flights_current
UNION
SELECT id, departed_flag
FROM flights_archive;
```

Или выбрать другой общий тип (например, текст).

```
SELECT id, created_at::text FROM flights_current
UNION
SELECT id, departed_flag::text FROM flights_archive;
```

3. Нельзя фильтровать общий результат без подзапроса.

```
-- ❌ Так фильтр применится только к последнему множеству
SELECT ... FROM A
UNION ALL
SELECT ... FROM B
WHERE ...;
```

Как надо?

```
SELECT *
FROM (
    SELECT ... FROM A
    UNION ALL
    SELECT ... FROM B
) t
WHERE ...;      -- фильтр на весь итог
```

4. Неверный ORDER BY (ссылка на имя колонки, которого нет в итоговом результате).

```
-- ❌ Не используется alias
SELECT name AS geo_name FROM w_country
UNION ALL
SELECT name FROM w_city
ORDER BY name;    -- name не существует в итоговом наборе
```

Как надо?

```
SELECT name AS geo_name FROM w_country
UNION ALL
SELECT name FROM w_city
ORDER BY geo_name;
```

Заключение

Операции над множествами позволяют объединять, сравнивать и вычитывать наборы данных. Знание правил их применения, особенностей сортировки и совместимости типов поможет писать эффективные и корректные запросы.

- UNION ALL — по умолчанию (быстрее чем UNION).
- UNION — когда нужна дедупликация и у нас небольшой набор данных.
- INTERSECT — общие элементы.
- EXCEPT — различия.

Комбинируй их с фильтрацией, сортировкой и группировкой, помня о правилах: фильтровать/группировать общий результат — только снаружи (подзапросом), сортировать — внутри обязательно с помощью скобок, а общий результат в конце вне скобок. Следуй требованиям совместимости типов, проверяй типовые ошибки, и твои запросы будут и корректными, и быстрыми.